

O Método Simplex: Otimização Prática com Python

Sarpa, Mauricio
Engenharia da Computação
Fundação Hermínio Ometto
Araras, Brasil
sarpa@alunos.fho.edu.br

Da Cunha, Julio Cezar
Engenharia da Computação
Fundação Hermínio Ometto
Araras, Brasil
Juliocezar@alunos.fho.edu.br

Buzatto, Diogo
Engenharia da Computação
Fundação Hermínio Ometto
Araras, Brasil
diogobuzatto@alunos.fho.edu.br

Silva, Lucas Ferreira
Engenharia da Computação
Fundação Hermínio Ometto
Araras, Brasil
lucas.silva2958@alunos.fho.edu.br

Resumo—Este artigo apresenta um relatório exaustivo sobre o método Simplex, um algoritmo fundamental para a resolução de problemas de Programação Linear (PL). Partindo de uma introdução aos conceitos de otimização, o texto explora os fundamentos matemáticos que sustentam o algoritmo, incluindo sua interpretação geométrica e a formulação matricial. A metodologia do Simplex tabular é detalhada passo a passo, servindo como base para uma implementação em Python. Um estudo de caso prático sobre otimização de transporte é modelado e resolvido, com visualização dos resultados.

Index Terms—Método Simplex, Programação Linear, Otimização, Python, Pesquisa Operacional.

I. INTRODUÇÃO À PROGRAMAÇÃO LINEAR E AO MÉTODO SIMPLEX

A. O Desafio da Otimização no Mundo Moderno

A otimização é um pilar central na tomada de decisões estratégicas em um mundo caracterizado pela abundância de dados e pela escassez de recursos. Em sua essência, otimizar significa encontrar a melhor solução possível para um problema, dadas certas limitações ou condições. A modelagem matemática surge como uma ferramenta indispensável nesse contexto, permitindo que problemas complexos do mundo real sejam representados de forma simplificada e estruturada por meio de uma linguagem universal: a matemática [1]. Essa abordagem possibilita a análise de diferentes cenários de forma mais rápida e econômica do que a experimentação prática, oferecendo um suporte robusto ao processo decisório em áreas tão diversas quanto logística, finanças, produção industrial e engenharia [1].

B. Programação Linear: Uma Ferramenta para a Tomada de Decisão

Dentro do vasto campo da otimização matemática, a Programação Linear (PL) destaca-se como uma das técnicas mais importantes e bem estudadas [3]. Um problema de PL consiste em encontrar o valor máximo ou mínimo de uma função linear, denominada **função objetivo**, que está sujeita a um conjunto de restrições lineares na forma de igualdades ou desigualdades [3]. É crucial entender que o termo "programação" neste contexto não se refere à programação de computadores, mas sim ao planejamento e à alocação ótima de recursos.

Os componentes fundamentais de um modelo de Programação Linear são:

- **Função Objetivo:** Uma expressão matemática linear que define a quantidade a ser otimizada. Por exemplo, em um contexto empresarial, pode ser a maximização do lucro ou a minimização dos custos de produção [5]. Matematicamente, é expressa como $Z = c_1x_1 + c_2x_2 + \dots + c_nx_n$.
- **Variáveis de Decisão:** São as incógnitas do problema, representadas por $x_1, x_2, \dots, x_n$. Elas quantificam as decisões a serem tomadas, como a quantidade de cada produto a ser fabricado ou o montante a ser investido em diferentes ativos [5].
- **Restrições:** São um conjunto de equações ou inequações lineares que representam as limitações do sistema, como disponibilidade de matéria-prima, capacidade de armazenamento, horas de mão de obra ou demanda de mercado

[5].

C. O Surgimento e a Importância do Método Simplex

Para solucionar problemas de PL com mais de duas ou três variáveis de decisão, onde métodos gráficos se tornam impraticáveis, é necessário um algoritmo mais robusto [?]. O **método Simplex**, desenvolvido por George Dantzig em 1947, foi um marco na história da programação matemática e simboliza o início da era moderna da otimização [2]. Considerado um dos algoritmos mais importantes do século XX, o Simplex é um procedimento numérico, iterativo e matricial projetado para encontrar sistematicamente a solução ótima de um modelo de PL [4].

A relevância do método Simplex transcende a teoria acadêmica, sendo uma ferramenta ativamente empregada para gerar valor econômico e eficiência na indústria brasileira. Uma análise da produção científica nacional revela uma forte conexão entre a teoria da Pesquisa Operacional e a prática da engenharia [11]. Estudos de caso demonstram sua aplicação bem-sucedida em diversos setores: na maximização do uso de matéria-prima em uma fábrica de semicondutores na Zona Franca de Manaus, resultando em uma redução de 92% no desperdício [8]; na otimização do mix de produção (embora em panificação, o princípio é similar ao de laticínios) [6]; e na melhoria de processos em microempresas do setor têxtil [11]. Essa ponte entre a academia e a indústria evidencia que a maestria do método Simplex e sua implementação computacional em linguagens como Python é uma habilidade de alto valor e aplicabilidade direta no mercado brasileiro.

II. FUNDAMENTOS MATEMÁTICOS DO ALGORITMO SIMPLEX

A eficácia do método Simplex está ancorada em uma base matemática sólida que conecta conceitos de álgebra linear e geometria. Para que o algoritmo possa ser aplicado, o problema de PL deve ser primeiramente convertido para um formato padronizado.

A. A Forma Padrão (FP)

O método Simplex opera sobre problemas de PL que estão na chamada **Forma Padrão** (ou Forma Canônica). Esta forma possui características específicas que facilitam o início do procedimento algorítmico [4]. Um problema de PL está na Forma Padrão se satisfaz as seguintes condições:

- 1) É um problema de **maximização**.
- 2) Todas as restrições são equações de igualdade (com exceção das restrições de não negatividade).
- 3) O lado direito de cada restrição (termo independente b_i) é não negativo ($b_i \geq 0$).
- 4) Todas as variáveis de decisão são não negativas ($x_j \geq 0$).

Qualquer problema de PL pode ser convertido para a Forma Padrão através das seguintes transformações:

- Um problema de minimização de uma função Z é equivalente à maximização de $-Z$ [6].
- Uma restrição de desigualdade do tipo $\sum a_{ij}x_j \leq b_i$ é convertida em uma igualdade pela adição de uma **variável de folga**.
- Uma restrição de desigualdade do tipo $\sum a_{ij}x_j \geq b_i$ é convertida em uma igualdade pela subtração de uma **variável de excesso**.
- Uma variável de decisão x_j que é "livre em sinal" pode ser substituída pela diferença de duas novas variáveis não negativas: $x_j = x'_j - x''_j$, onde $x'_j \geq 0$ e $x''_j \geq 0$.

B. Variáveis de Folga e Excesso

As **variáveis de folga** são um conceito central na preparação de um problema para o Simplex. Elas são adicionadas a restrições do tipo "menor ou igual a" ($\leq$) para transformá-las em igualdades. Fisicamente, uma variável de folga representa a quantidade de um recurso que não foi utilizada [10]. Por exemplo, a restrição $2x_1 + 3x_2 \leq 100$ (horas de máquina) é convertida para $2x_1 + 3x_2 + s_1 = 100$, onde $s_1 \geq 0$ é a variável de folga que representa o número de horas de máquina ociosas [5].

De forma análoga, as **variáveis de excesso** são subtraídas de restrições do tipo "maior ou igual a" ($\geq$) para convertê-las em igualdades. Elas representam a quantidade pela qual o lado esquerdo da inequação excede o lado direito. A restrição $x_1 + x_2 \geq 50$ (demanda mínima) torna-se $x_1 + x_2 - e_1 = 50$, onde $e_1 \geq 0$ é a variável de excesso.

III. O MÉTODO SIMPLEX TABULAR: UM GUIA PASSO A PASSO

O método Simplex tabular, é uma representação matricial que organiza os coeficientes do problema de PL e facilita a aplicação mecânica das iterações do algoritmo. Ele é uma forma concisa de acompanhar as trocas de variáveis e a evolução da solução [5].

A. Construção do Tableau Inicial

Para um problema de maximização na Forma Padrão, o tableau inicial é construído da seguinte forma:

- 1) A função objetivo, por exemplo $Z = c_1x_1 + c_2x_2$, é reescrita como $Z - c_1x_1 - c_2x_2 = 0$.
- 2) Cria-se uma tabela. As colunas correspondem a cada variável de decisão (x_i), cada variável de folga (s_j) e ao lado direito das equações (RHS).
- 3) As linhas correspondem a cada restrição. A última linha (linha Z) representa a função objetivo.
- 4) Os coeficientes das restrições são preenchidos nas linhas correspondentes.

- 5) Na linha Z, inserem-se os coeficientes da função objetivo reescrita ($-c_i$). O valor inicial de Z (geralmente 0) é colocado na coluna RHS da linha Z.

A solução básica inicial geralmente consiste nas variáveis de folga, que formam uma matriz identidade dentro do tableau [5].

B. O Processo Iterativo

O algoritmo Simplex é um processo iterativo que se repete até que a solução ótima seja encontrada. Cada iteração consiste nos seguintes passos:

Passo 1: Verificação da Condição de Otimalidade A solução atual é ótima se todos os coeficientes na linha Z forem não negativos (para maximização) [5]. Se sim, o algoritmo termina.

Passo 2: Escolha da Variável de Entrada (Coluna Pivô) Se a condição não for atendida, a variável associada ao coeficiente mais negativo (menor valor) na linha Z é escolhida para entrar na base. Esta é a **coluna pivô** [5].

Passo 3: Escolha da Variável de Saída (Linha Pivô) Realiza-se o **teste da razão mínima**. Para cada linha das restrições, divide-se o valor da coluna RHS pelo coeficiente correspondente na coluna pivô (apenas denominadores positivos). A linha com a menor razão não negativa é a **linha pivô** [5].

Passo 4: Operação de Pivoteamento O elemento na interseção da coluna pivô e linha pivô é o **elemento pivô**. O objetivo é transformar a coluna pivô em uma coluna da matriz identidade usando operações elementares de linha (eliminação de Gauss-Jordan) [5].

Passo 5: Repetição Após o pivoteamento, o processo retorna ao Passo 1.

A Tabela I demonstra este processo para o problema: Maximizar $Z = 3x_1 + 5x_2$ sujeito a $x_1 \leq 4$, $2x_2 \leq 12$ e $3x_1 + 2x_2 \leq 18$.

C. Interpretando a Solução Final

No tableau final (Tabela I):

- As variáveis básicas são $s_1 = 2$, $x_2 = 6$, $x_1 = 2$.
- As variáveis não básicas são $s_2 = 0$, $s_3 = 0$ [5].
- A solução ótima é $x_1 = 2$, $x_2 = 6$.
- O valor máximo é $Z = 36$ [5], [10].

D. Casos Especiais: O Método das Duas Fases

Quando restrições do tipo $\geq$ ou $=$ existem, a base inicial de folga não é viável. Nesses casos, **variáveis artificiais** são adicionadas para criar uma base inicial [6], [10]. O **Método das Duas Fases** é aplicado:

- **Fase I:** Minimiza-se a soma das variáveis artificiais. Se o mínimo for zero, uma SBV para o problema original foi encontrada, e passa-se para a Fase II. Se o mínimo for maior que zero, o problema original é inviável [9], [10].

Tabela I
RESOLUÇÃO DE EXEMPLO PELO MÉTODO SIMPLEX TABULAR

Tableau Inicial (Iteração 0)							
Base	x_1	x_2	s_1	s_2	s_3	RHS	Razão
s_1	1	0	1	0	0	4	-
s_2	0	2	0	1	0	12	$12/2 = 6$
s_3	3	2	0	0	1	18	$18/2 = 9$
Z	-3	-5	0	0	0	0	

Tableau Após Iteração 1							
Base	x_1	x_2	s_1	s_2	s_3	RHS	Razão
s_1	1	0	1	0	0	4	$4/1 = 4$
x_2	0	1	0	0.5	0	6	-
s_3	3	0	0	-1	1	6	$6/3 = 2$
Z	-3	0	0	2.5	0	30	

Tableau Após Iteração 2 (Ótimo)							
Base	x_1	x_2	s_1	s_2	s_3	RHS	Razão
s_1	0	0	1	1/3	-1/3	2	
x_2	0	1	0	1/2	0	6	
x_1	1	0	0	-1/3	1/3	2	
Z	0	0	0	1.5	1	36	

- **Fase II:** O tableau final da Fase I (sem as variáveis artificiais) é usado como ponto de partida. O Simplex padrão é aplicado à função objetivo original [9].

IV. OTIMIZAÇÃO LINEAR COM PYTHON

Para aplicações práticas, é mais eficiente usar bibliotecas que separam a **modelagem** da **resolução**. O usuário descreve o problema, e a biblioteca invoca um **solver** de alto desempenho.

A. Por que Usar Bibliotecas?

Bibliotecas como 'SciPy' e 'PuLP' são interfaces para solvers poderosos (CBC, GLPK, Gurobi, CPLEX), que são motores computacionais otimizados em C++ ou Fortran, capazes de resolver problemas de larga escala com rapidez e estabilidade numérica [2].

B. SciPy.optimize.linprog: Otimização Baseada em Matrizes

A função 'linprog' do SciPy é a ferramenta padrão no ecossistema científico do Python. Ela requer que o problema seja fornecido em formato matricial.

É fundamental notar que 'linprog', por padrão, resolve problemas de **minimização**. Para maximizar Z , deve-se minimizar $-Z$, passando o vetor 'c' com sinais trocados.

```
from scipy.optimize import linprog
import numpy as np

# Maximizar Z = 3x1 + 5x2 => Minimizar -Z = -3x1 - 5x2
```

```

5 c_scipy = np.array([-3, -5])
6
7 # Restrições (todas já são <=)
8 A_ub_scipy = np.array([[1, 0], [0, 2], [3, 2]])
9 b_ub_scipy = np.array([4, 12, 18])
10
11 # Limites (x1 >= 0, x2 >= 0)
12 x1_bounds = (0, None)
13 x2_bounds = (0, None)
14
15 # Resolver o problema (method='Simplex' é o padrão
    atual)
16 resultado = linprog(c_scipy, A_ub=A_ub_scipy,
17                      b_ub=b_ub_scipy,
18                      bounds=[x1_bounds, x2_bounds],
19                      method='Simplex')
20
21 if resultado.success:
22     print("--- Solução Ótima (SciPy) ---")
23     # 0 valor da função é negativo
24     print(f"Valor Z máximo: {-resultado.fun:.2f}")
25     print(f"x1 = {resultado.x[0]:.2f}")
26     print(f"x2 = {resultado.x[1]:.2f}")
27 else:
28     print(f"Solução não encontrada: {resultado.
        message}")

```

C. PuLP: Modelagem Algébrica Intuitiva

PuLP permite definir problemas de PL usando uma sintaxe próxima da notação matemática, tornando o código mais legível [2].

```

1 import pulp
2
3 # 1. Inicializar o modelo
4 modelo = pulp.LpProblem("Exemplo_Maximizacao",
5                          pulp.LpMaximize)
6
7 # 2. Definir as variáveis
8 x1_pulp = pulp.LpVariable('x1', lowBound=0)
9 x2_pulp = pulp.LpVariable('x2', lowBound=0)
10
11 # 3. Adicionar a função objetivo
12 modelo += 3*x1_pulp + 5*x2_pulp, "Funcao_Objetivo"
13
14 # 4. Adicionar as restrições
15 modelo += x1_pulp <= 4, "Restricao_1"
16 modelo += 2*x2_pulp <= 12, "Restricao_2"
17 modelo += 3*x1_pulp + 2*x2_pulp <= 18, "Restricao_3"
18
19 # Resolver o problema (usa o solver CBC por padrão)
20 modelo.solve()
21
22 print("--- Solução Ótima (PuLP) ---")
23 print(f"Status: {pulp.LpStatus[modelo.status]}")
24 print(f"Valor Z máximo: {pulp.value(modelo.objective)
    :.2f}")
25 print(f"x1 = {pulp.value(x1_pulp):.2f}")
26 print(f"x2 = {pulp.value(x2_pulp):.2f}")

```

A sintaxe do PuLP é mais declarativa, sendo preferida para prototipagem rápida.

V. ESTUDO DE CASO: OTIMIZAÇÃO DE UM PROBLEMA DE TRANSPORTE

Este estudo de caso aborda o clássico **Problema de Transporte**, que busca determinar o plano de distribuição de menor custo de origens para destinos, respeitando ofertas e demandas.

A. Formulação do Problema

Considere 2 fábricas e 3 centros de distribuição.

- Oferta (Capacidade):

- Fábrica 1 (BH): 1000 unidades
- Fábrica 2 (CP): 1500 unidades

- Demanda (Necessidade):

- CD 1 (RJ): 800 unidades
- CD 2 (SP): 900 unidades
- CD 3 (CT): 600 unidades

Custos de Transporte (R\$/unidade):

De / Para	CD 1 (RJ)	CD 2 (SP)	CD 3 (CT)
Fábrica 1 (BH)	5	4	7
Fábrica 2 (CP)	6	3	5

Modelagem Matemática:

Seja x_{ij} a quantidade transportada da fábrica i para o CD j .

Função Objetivo (Minimizar Custo Total):

$$\min Z = 5x_{11} + 4x_{12} + 7x_{13} + 6x_{21} + 3x_{22} + 5x_{23} \quad (1)$$

Restrições de Oferta ($\leq$):

$$x_{11} + x_{12} + x_{13} \leq 1000 \quad (\text{Fábrica 1}) \quad (2)$$

$$x_{21} + x_{22} + x_{23} \leq 1500 \quad (\text{Fábrica 2}) \quad (3)$$

Restrições de Demanda ($\geq$):

$$x_{11} + x_{21} \geq 800 \quad (\text{CD 1}) \quad (4)$$

$$x_{12} + x_{22} \geq 900 \quad (\text{CD 2}) \quad (5)$$

$$x_{13} + x_{23} \geq 600 \quad (\text{CD 3}) \quad (6)$$

Não Negatividade: $x_{ij} \geq 0$.

B. Resolução com Python e Análise dos Resultados

A estrutura matricial do problema é ideal para 'scipy.optimize.linprog'.

```

1 import numpy as np
2 from scipy.optimize import linprog
3
4 # Coeficientes da função objetivo (c)
5 # Ordem: x11, x12, x13, x21, x22, x23
6 c_transporte = np.array([5, 4, 7, 6, 3, 5])
7
8 # Restrições de desigualdade (A_ub @ x <= b_ub)
9 # Convertendo demanda (>=) para (<=) multiplicando
    por -1
10 A_transporte = np.array([
11     # Oferta (<=)
12     [1, 1, 1, 0, 0, 0], # Fábrica 1
13     [0, 0, 0, 1, 1, 1], # Fábrica 2
14     # Demanda (convertida de >= para <=)
15     [-1, 0, 0, -1, 0, 0], # Demanda CD 1
16     [0, -1, 0, 0, -1, 0], # Demanda CD 2
17     [0, 0, -1, 0, 0, -1] # Demanda CD 3
18 ])
19
20 b_transporte = np.array([
21     1000, # Oferta F1
22     1500, # Oferta F2
23     -800, # Demanda CD 1
24     -900, # Demanda CD 2

```

```

25     -600 # Demanda CD 3
26 ])
27
28 # Limites (todas as variáveis >= 0)
29 bounds = [(0, None) for _ in range(len(c_transporte)
30         )]
31
32 # Resolver
33 resultado_transporte = linprog(c_transporte,
34                                A_ub=A_transporte,
35                                b_ub=b_transporte,
36                                bounds=bounds,
37                                method='Simplex')
38
39 if resultado_transporte.success:
40     print("\n--- Plano de Transporte Ótimo ---")
41     print(f"Custo Total Mínimo: R$ {
42         resultado_transporte.fun:.2f}")
43     solucao = resultado_transporte.x.reshape(2, 3)
44     print("Plano de Distribuição (unidades):")
45     print("      | CD 1 (RJ) | CD 2 (SP) | CD 3
46           (CT)")
47     print(f"Fáb. 1 (BH) | {solucao[0, 0]:9.0f} | {
48         solucao[0, 1]:9.0f} | {solucao[0, 2]:9.0f}")
49     print(f"Fáb. 2 (CP) | {solucao[1, 0]:9.0f} | {
50         solucao[1, 1]:9.0f} | {solucao[1, 2]:9.0f}")

```

C. Visualização dos Resultados

A comunicação eficaz dos resultados é crucial. A visualização de dados serve como ponte entre a análise complexa e a inteligência de negócios [2]. O código Matplotlib para gerar o gráfico de barras foi omitido, mas a Figura 1 representa a saída visual esperada em Gráfico de barras e na Figura 2 esta uma representação em grafos, mostrando como será a distribuição de uma forma mais dinâmica e visual.

VI. CONCLUSÃO

Este artigo percorreu o método Simplex, desde seus fundamentos teóricos até sua aplicação prática em Python. A construção de um solver "do zero" com NumPy desmistificou o algoritmo, mas também evidenciou as limitações que justificam o uso de bibliotecas profissionais como 'SciPy' e 'PuLP'.

Ficou claro que, para aplicações práticas, estas ferramentas são preferenciais, permitindo ao analista focar na modelagem. A sintaxe algébrica do PuLP destaca-se pela clareza. O estudo de caso do Problema de Transporte consolidou a aplicação e ressaltou a importância da visualização de dados para comunicar a solução. O método Simplex, combinado com Python, permanece uma ferramenta de otimização atemporal, impulsionando a eficiência na indústria e academia brasileira [2], [11].

REFERÊNCIAS

- [1] E. L. Andrade, *Introdução à pesquisa operacional: métodos e modelos para análise de decisões*, 4ª ed. Rio de Janeiro: LTC, 2009.
- [2] D. F. Cordeiro *et al*., "Otimização e visualização de dados com Python: um estudo de caso de roteamento de veículos," em *Simpósio de Engenharia de Produção*, Catalão, 2022.

- [3] F. S. M. Lima, "Proposta de Otimização do Processo Produtivo em uma Microempresa do Setor Têxtil: Aplicação do Método Simplex," *Revista Ibero-Americana de Gestão, Inovação e Empreendedorismo*, vol. 2, n. 1, pp. 1-15, 2023.
- [4] A. F. U. S. Macambira *et al*., *Programação Linear*. João Pessoa: Editora da UFPB, 2016.
- [5] J. L. Menezes Neto e W. A. B. Silva, "Resolução de problemas em programação linear via método simplex por tabelas," *Revista De Matemática Da UFOP*, vol. 3, n. 3, pp. 1-10, 2023.
- [6] D. A. Milhomem *et al*., "Aplicação da Programação Linear e do Método Simplex para a otimização de pães em uma empresa de panificação," em *Encontro Nacional de Engenharia de Produção - ENEGEP*, Fortaleza, 2015.
- [7] T. F. Okiishi e L. F. R. Souza, "Método Simplex aplicado à minimização dos custos de transporte de uma rede de farmácias," *Revista FIRA*, Avaré, vol. 3, n. 1, pp. 15-21, 2011.
- [8] A. S. Pereira *et al*., "Aplicação do método simplex para maximizar o uso da matéria-prima na fabricação de semicondutores e reduzir o tempo de cálculo na preparação dos lotes para produção," em *Simpósio de Engenharia de Produção*, Manaus, 2022.
- [9] R. M. Schneider, "Uma introdução à programação linear," Trabalho de Conclusão de Curso, Dept. Mat., Univ. Federal de Santa Catarina, Florianópolis, 2012.
- [10] A. C. M. Silva e M. J. F. Souza, "Programação linear: método de otimização simplex e software OTIMIZA," *Revista Espacios*, vol. 38, n. 60, p. 4, 2017.
- [11] D. R. Silva *et al*., "Mapeamento de artigos que utilizaram o método simplex para resolução de problemas de programação linear," em *Encontro de Iniciação Científica e Pós-Graduação do IFNMG*, Montes Claros, 2018.
- [12] D. Buzatto, M. Sarpa, J. C. Da Cunha, e L. F. Silva, "Otimização de Rotas de Distribuição: Implementação do Método Simplex em Python," Repositório GitHub. [Online]. Disponível: <https://github.com/dbuzatto/Otimizacao-de-Rotas-de-Distribuicao>.

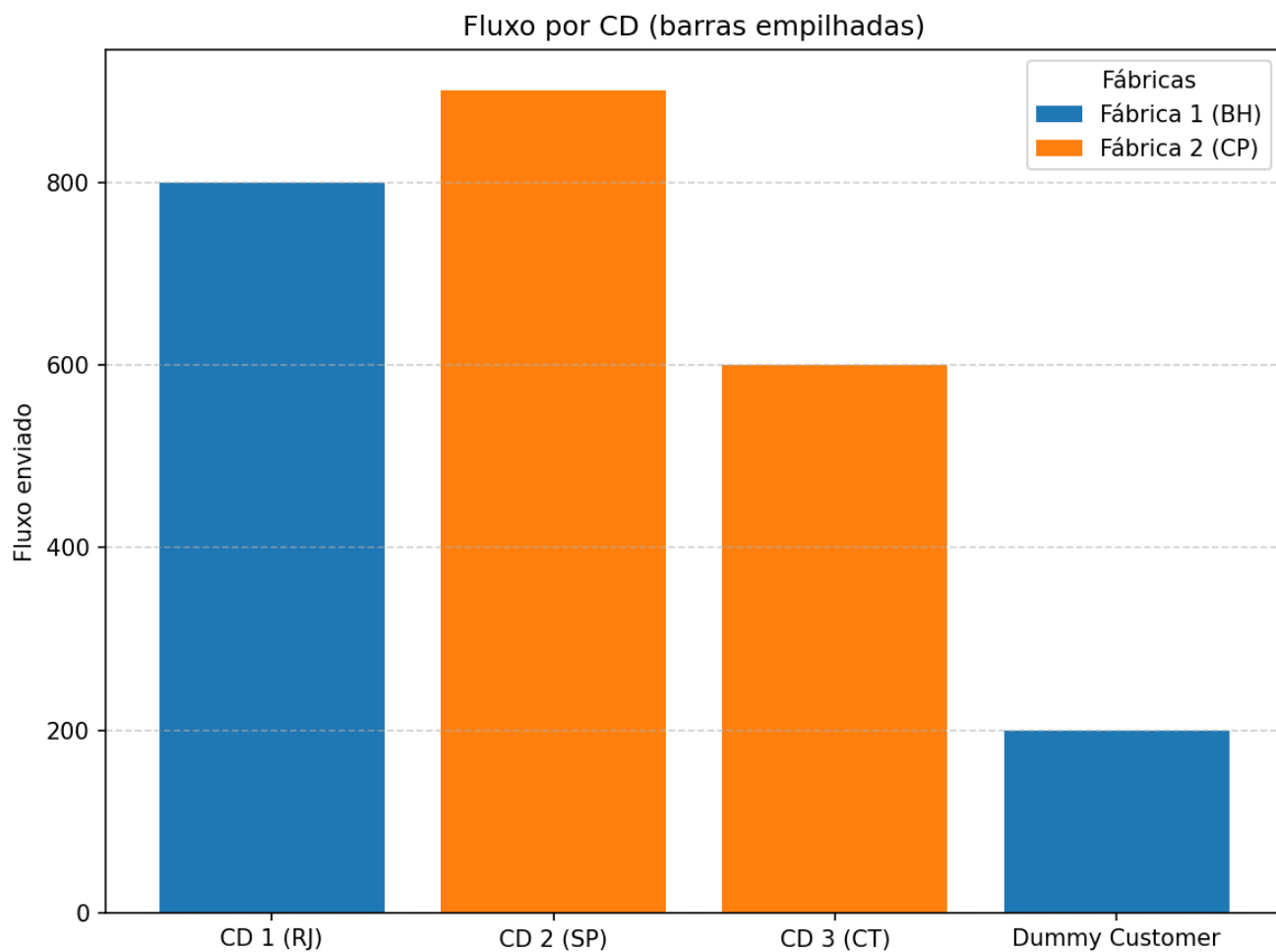


Figura 1. Visualização do Plano de Distribuição Ótimo (Gráfico de Barras Empilhadas). Esta figura ilustra como cada CD é abastecido pelas fábricas, conforme a solução ótima.

Grafo bipartido de fluxos

